

COMP 6721 - Progress report - Group A

A. Introduction and problem statement

Malware detection has become increasingly critical as the frequency and sophistication of digital threats continue to rise. The traditional approaches of malware detection, such as block lists for file extensions, static signatures, and dynamic analysis are limited in their ability to keep up with the evolving tactics used by attackers [1]. Malware authors frequently update their techniques, rendering conventional methods less effective over time [2]. As a result, modern malware detection requires more advanced and adaptive technologies that can anticipate and respond to these changes without relying solely on predefined rules. In this situation, Artificial intelligence (AI), particularly deep learning, offers a transformative approach [3].

Nowadays, research regarding deep learning for malware detection can be divided into four areas which are behavior analysis detection, anomaly detection, signature-based detection, and automated classification[4]. Behavior analysis focuses on using deep learning to detect malware actions and behavioral patterns on machines. Anomaly detection focuses on using deep learning to create a model of "normal" behavior to highlight potential threats[4]. Signature-based detection uses deep learning to distinguish possible malware files with signatures. Automated classification uses deep learning to automate the detection and classification of possible malware files. Even though automated classification has been proven that it can detect malware efficiently[5], there still exists one issue regarding it, which is automated classification requires a lot of resources to deal with complex data. In this project, we focus on automated classification using Portable Executable(PE) files in byteplot image format, which may help prevent malware initially and reduce workload for other methods. Furthermore, by converting binary files into structured byteplots, we can reduce data complexity and make the classification process more resource-efficient. This approach has great potential to resolve the computational burden typically associated with malware detection.

In this project, we are developing a malware detection system using CNNs (ResNet-18, EfficientNet-B0, and DenseNet-121) to analyze image-based representations of malware files, leveraging PE byteplot images. We are training and evaluating these models on three malware datasets, using data preprocessing techniques to balance classes and ensure consistent image quality. Our training involved systematic tuning of hyperparameters, with the Adam optimizer and cross-entropy loss yielding strong performance. We will evaluate model success based on test accuracy, convergence rate, and test loss, with future steps involving transfer learning to further validate and enhance performance on larger datasets.

B. Proposed Methodologies

In this project, we aim to leverage Convolutional neural networks(CNNs) to develop a malware detection system that analyzes image-based representations of malware files. By using Portable Executable (PE) files in byteplot image format, our approach allows CNNs to interpret malware data. CNNs are excellent at identifying image patterns, making it possible for the model to detect subtle similarities and differences in samples that traditional methods may miss. To train and evaluate this system, we employed three diverse malware image datasets from Kaggle. The large dataset is the Blended Malware Image Dataset [6], which includes 9,868 PNG images across 31 malware classes, each at a resolution of 224x224 pixels. Secondly, we are using the Malware Images dataset[7] as the medium dataset, containing 125,004 images in 17 classes. Although it is skewed toward a few specific classes, preprocessing steps will help balance it as our moderate dataset. Finally, the Malware-Byteplot Images dataset[8], offers a binary classification of 60,337 images labeled as "Benign" and "Malicious." This dataset, although slightly skewed towards the malicious class, will serve as our small dataset.

Regarding the choice of CNN, this project uses ResNet[9], EfficientNet[10], and DenseNet[11] as the pre-trained models. ResNet utilizes residual connections, which help in learning deeper networks and mitigating the vanishing gradient problem. We are using ResNet-18, a simpler variation of the architecture, to strike a balance between depth and efficiency. EfficientNet uses compound scaling, where depth, width, and resolution are scaled together in a balanced manner, which allows it to achieve better performance with fewer parameters. EfficientNet-B0 also employs compound scaling, optimizing width, depth, and resolution. It is a basic but efficient model in the EfficientNet family. DenseNet, known for its densely connected layers, strengthens feature propagation and reuse, which enhances detection accuracy. DenseNet-121 is being used for its simplicity and efficiency.

As mentioned in the previous paragraph, some datasets are skewed toward a few specific classes. To address this and prevent it from negatively impacting the training process and final CNN performance, this project incorporates a data preprocessing pipeline. The first step involves resizing all images to 224x224, ensuring consistent image dimensions and avoiding errors related to varying sizes. The second step is to calculate the sample count per class, find the average number of samples across all classes, and then balance the sample count between classes. This balancing is achieved using undersampling and oversampling. If some of the classes have more samples than the average number of samples, the pipeline will perform undersampling by se-

lecting a random subset of the images to match the target count. In some of the classes have fewer samples than the average number of samples, the pipeline will perform over-sampling by randomly duplicating a subset of the images and performing augmentation action until the target count is reached. Augmentation includes random rotation, width shift, height shift, random adjustments on the brightness range, and random horizontal flip. Balancing the dataset through these augmentations helps prevent poor generalization, especially for minority classes, and ensures that the model can learn from a more representative distribution of data[12]. The third step of this pipeline is to normalize all sample image pixel values from between 0 and 255 to between 0 and 1. This normalization helps reduce computation time during training and reduces variability, leading to more consistent activations and gradient distributions across the network[13]. The last step is to split the all samples into three sets i.e., Training, Validation and Testing. The percentage is 70, 10, 20 for each set respectively. A validation set was used during the training process to evaluate the models' performance after each epoch. The preprocessed data is saved to perform the further model training on.

C. Attempts at solving the problem

We start the attempts to solve this problem by first implementing and training the ResNet-18 for the medium dataset. Our training strategy starts by experimenting with different hyperparameters, including batch size, number of epochs, learning rate, loss functions, and optimizers. We first identify the best-performing loss function and optimizer, then fine-tune the remaining hyperparameters based on convergence patterns, training and validation accuracy, and the confusion matrix. With broad exploration, first discovered that the adaptive moment estimation (Adam) optimizer and cross-entropy loss are generally performing better than others for our problem. Then we temporarily fixed the learning rate to 0.001 and tested on different batch numbers and epoch numbers. We went through 7, 10, and 20 for epochs, and 16, 32, and 64 for batches. After those mix-and-match trials, we found out that the batch size equal to 32 and the epoch around 10 would give us fast convergence and great training accuracy with our machine. During training, we noticed with the increasing number of epochs, the training accuracy would increase while the validation accuracy would remain constant around 74% with increasing validation loss after a few epochs which is a clear sign of overfitting. We first thought it was occurring because we were using a large epoch number. However, even if we reduce the epoch back down to 7, the issue persists. We applied various techniques to resolve it, which included applying L-2 regularization and decreasing the learning rate to 0.0001. For regularization, we tried adding weight decay in the training to prevent the model from developing large weights

for certain classes, but it did not resolve the overfitting issue. Therefore, we plotted out the confusion matrix (Figure 1) and discovered that RecordBreaker, Formbook, RedLineStealer, and NanoCore classes in our medium dataset have similar features and these classes are causing the overfitting. We removed the classes Formbook, RedLineStealer and NanoCore, and added Denabot. Hence, the model did not show overfitting on training the new processed data. The new Confusion Matrix (Figure 2) turned out to be perfect, and the training and validation accuracy turned resulted above 90% with no significant increased losses. We also plotted out the training and validation loss to epoch, and training and validation accuracy on epoch. As shown in Figure 3, the model starts to converge at epoch 5. It reached the best performance at epoch 9 with 99.42% training accuracy and 92.67% validation accuracy.

We also trained EfficientNet-B0 on the same medium dataset with the same strategy. We discovered epoch 10 and batch value 32 with cross-entropy loss and Adam optimizer is performing the best. As a result, it reached 92.61% accuracy and 0.3187 loss value on the test set. Based on Figure 5, we can see the model starts to converge at epoch 10, which is slightly later than ResNet-18. In epoch 1-, this model reached its highest training accuracy 96.95% with a validation accuracy of 93.28%. Compared with the ResNet-18 model, it has slightly lower training accuracy but has a better validation accuracy. In Figure 6 and 7 you can find out the resultant screenshots of running our program which are shown on both the models ResNet-18 and EfficientNet-B0 respectively. It contains Training, Validation and Test Accuracies and its Loss mentioned in it.

D. Future Steps

For the next steps, this project will complete the training of DenseNet using the medium dataset. Following this, the strategy outlined in the previous section will be applied to all CNNs, using both small and large datasets. Additionally, transfer learning will be employed with the two best-performing models trained on the medium dataset, and these models will be fine-tuned for the larger dataset. Given that transfer learning is known to reduce training time and enhance performance on smaller datasets [14], the models trained on the medium dataset will be fine-tuned on the larger dataset. Although there is no class overlap between these two datasets, transfer learning is expected to be effective, as the pre-trained models will leverage general features learned from the medium dataset. This will help accelerate convergence and improve performance. Simultaneously, this project will continue to refine the model with the lowest accuracy on the test data, iterating towards improvement until the end. Lastly, this project will compare all models with their performance and accuracy on test data and form a conclusion in the final report.

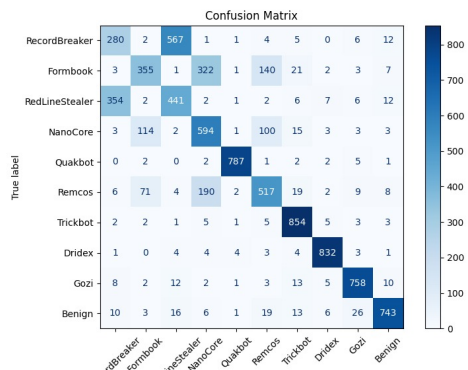


Figure 1. Confusion Matrix from the overfitting ResNet-18 model.

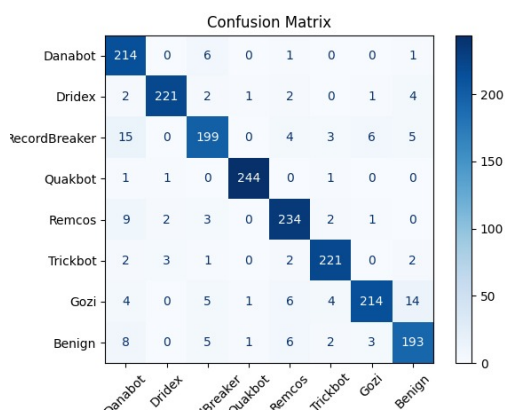


Figure 2. Confusion Matrix from the Fixed ResNet-18 model.

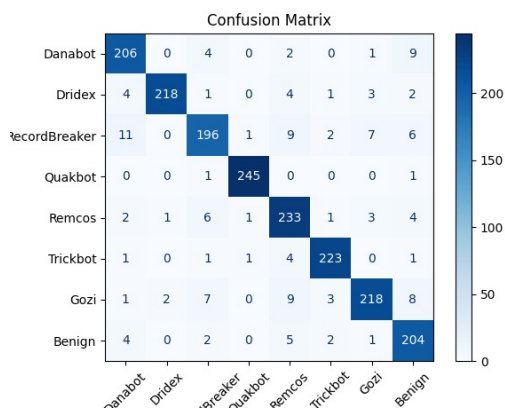


Figure 3. Confusion Matrix from the EfficientNet-B0 model.

References

- [1] Djenna A, Bouridane A, Rubab S, Marou IM. *Artificial Intelligence-Based Malware Detection, Analysis, and Mitigation*. Symmetry. 2023; 15(3):677. <https://doi.org/10.3390/sym15030677>

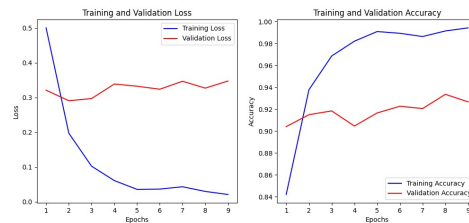


Figure 4. Training and validation loss and training and validation accuracy respect to epoch with ResNet-18 model.

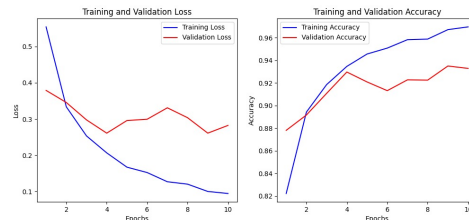


Figure 5. Training and validation loss and training and validation accuracy respect to epoch with EfficientNet-B0 model.

```
(malware-detection) C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\python src\models\train_ResNet.py
Class distribution in X_test based on y_test labels:
Class 0: 222 samples in X_test
Class 1: 233 samples in X_test
Class 2: 232 samples in X_test
Class 3: 247 samples in X_test
Class 4: 251 samples in X_test
Class 5: 231 samples in X_test
Class 6: 248 samples in X_test
Class 7: 218 samples in X_test
Class distribution in X_train based on y_train labels:
Class 0: 1665 samples in X_train
Class 1: 1678 samples in X_train
Class 2: 1628 samples in X_train
Class 3: 1622 samples in X_train
Class 4: 1637 samples in X_train
Class 5: 1614 samples in X_train
Class 6: 1658 samples in X_train
Class 7: 1677 samples in X_train
Epoch 1/9, Train Loss: 0.5084, Train Accuracy: 0.8421, Val loss: 0.3211, Val Accuracy: 0.9041
Epoch 2/9, Train Loss: 0.1972, Train Accuracy: 0.9377, Val loss: 0.2907, Val Accuracy: 0.9158
Epoch 3/9, Train Loss: 0.1024, Train Accuracy: 0.9686, Val loss: 0.2971, Val Accuracy: 0.9184
Epoch 4/9, Train Loss: 0.0611, Train Accuracy: 0.9821, Val loss: 0.3188, Val Accuracy: 0.9046
Epoch 5/9, Train Loss: 0.0357, Train Accuracy: 0.9909, Val loss: 0.3225, Val Accuracy: 0.9166
Epoch 6/9, Train Loss: 0.0365, Train Accuracy: 0.9893, Val loss: 0.3239, Val Accuracy: 0.9227
Epoch 7/9, Train Loss: 0.0432, Train Accuracy: 0.9863, Val loss: 0.3468, Val Accuracy: 0.9205
Epoch 8/9, Train Loss: 0.0298, Train Accuracy: 0.9914, Val loss: 0.3271, Val Accuracy: 0.9238
Epoch 9/9, Train Loss: 0.0210, Train Accuracy: 0.9942, Val loss: 0.3475, Val Accuracy: 0.9267
Model saved to C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\trained models\malware_resnet18.pth
Test Loss: 0.2526, Test Accuracy: 92.45%
Unique classes in y_true: [0 1 2 3 4 5 6 7]
Unique classes in y_pred: [0 1 2 3 4 5 6 7]
shape==== 8
(malware-detection) C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\
```

Figure 6. Result of running ResNet-18 model program.

```
(malware-detection) C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\python src\models\train_EfficientNet.py
Class distribution in X_test based on y_test labels:
Class 0: 222 samples in X_test
Class 1: 233 samples in X_test
Class 2: 232 samples in X_test
Class 3: 247 samples in X_test
Class 4: 251 samples in X_test
Class 5: 231 samples in X_test
Class 6: 248 samples in X_test
Class 7: 218 samples in X_test
Class distribution in X_train based on y_train labels:
Class 0: 1665 samples in X_train
Class 1: 1678 samples in X_train
Class 2: 1628 samples in X_train
Class 3: 1622 samples in X_train
Class 4: 1637 samples in X_train
Class 5: 1614 samples in X_train
Class 6: 1658 samples in X_train
Class 7: 1677 samples in X_train
Epoch 1/10, Train Loss: 0.5539, Train Accuracy: 0.8222, Validation Loss: 0.3788, Validation Accuracy: 0.8788
Epoch 2/10, Train Loss: 0.3135, Train Accuracy: 0.9042, Validation Loss: 0.3555, Validation Accuracy: 0.8916
Epoch 3/10, Train Loss: 0.2533, Train Accuracy: 0.9185, Validation Loss: 0.2975, Validation Accuracy: 0.9107
Epoch 4/10, Train Loss: 0.2060, Train Accuracy: 0.9346, Validation Loss: 0.2618, Validation Accuracy: 0.9296
Epoch 5/10, Train Loss: 0.1673, Train Accuracy: 0.9455, Validation Loss: 0.2558, Validation Accuracy: 0.9388
Epoch 6/10, Train Loss: 0.1524, Train Accuracy: 0.9587, Validation Loss: 0.2994, Validation Accuracy: 0.9131
Epoch 7/10, Train Loss: 0.1582, Train Accuracy: 0.9582, Validation Loss: 0.2399, Validation Accuracy: 0.9227
Epoch 8/10, Train Loss: 0.1204, Train Accuracy: 0.9587, Validation Loss: 0.3036, Validation Accuracy: 0.9224
Epoch 9/10, Train Loss: 0.1083, Train Accuracy: 0.9678, Validation Loss: 0.2618, Validation Accuracy: 0.9349
Epoch 10/10, Train Loss: 0.0942, Train Accuracy: 0.9669, Validation Loss: 0.2923, Validation Accuracy: 0.9328
Model saved to C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\trained models\malware_efficientnetb0.pth
Test Loss: 0.3187, Test Accuracy: 92.61%
Unique classes in y_true: [0 1 2 3 4 5 6 7]
Unique classes in y_pred: [0 1 2 3 4 5 6 7]
shape==== 8
(malware-detection) C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\
```

Figure 7. Result of running EfficientNet-B0 model program.

- [2] Molinari S, Packer J. *MALWARE SCIENCE a Comprehensive Guide to Detection, Analysis, and Com-*

- pliance. Packt Publishing Ltd.; 2023. <https://www.oreilly.com/library/view/-/9781804618646/>
- [3] Wolsey, A. (2022). *The State-of-the-Art in AI-Based Malware Detection Techniques: A Review*. arXiv preprint arXiv:2210.11239. <https://arxiv.org/abs/2210.11239>
- [4] Dixon Natasha. (2023). *The Role of AI in Malware Detection and Prevention* MalwareBrains. Available online: <https://malwarebrains.com/ai-in-malware-detection/>
- [5] A. Almuqren, M. Frikha and A. Albuali. (2023). *Automated Malware Detection Based on a Machine Learning Algorithm*. 2023 IEEE Tenth International Conference on Communications and Networking (ComNet), Hammamet, Tunisia, 2023, pp. 1-12, doi: 10.1109/ComNet60156.2023.10366550. <https://ieeexplore.ieee.org/document/10366550>
- [6] Gaurav Pendharkar. (2022). *Blended Malware Image Dataset*. Kaggle. Available online: <https://www.kaggle.com/datasets/gauravpendharkar/blended-malware-image-dataset>
- [7] Walt. (2022). *Malware Images*. Kaggle. Available online: <https://www.kaggle.com/datasets/walt30/malware-images/>
- [8] Ajit Varma. (2024). *Malware-Byteplot Images*. Kaggle. Available online: <https://www.kaggle.com/datasets/ajit22/malware-byteplot-images>
- [9] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
- [10] Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks*. In International Conference on Machine Learning (pp.6105-6114). PMLR.
- [11] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). *Densely connected convolutional networks*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4700-4708).
- [12] Ghasemi A, Soleymani M, Asgarian F. (2022). *A novel deep learning-based approach for image classification using a hybrid feature extraction method*. Mathematics. 2022;14(11):2304. doi:10.3390/math14112304. Available from: <https://www.mdpi.com/2073-8994/14/11/2304>
- [13] Poulson JM, Wu YC, Nadiga BT. (2022). *Normalization effects on deep neural networks*. arXiv; 2022. Available from: <https://ar5iv.labs.arxiv.org/html/2209.01018>
- [14] Wang Y, Mantecon H, Weijer J, et al. (2021). *TransferI2I: Transfer Learning for Image-to-Image Translation from Small Datasets*. arXiv. Published May 13, 2021. Available from: <https://ar5iv.labs.arxiv.org/html/2105.06219>
- [15] Shehzer-Aurangzeb. (2021). *Malware-Detection-CNN*. Github. 2024. Available from: <https://github.com/Shehzer-Aurangzeb/Malware-Detection-CNN>